프론트엔드 자동화 CLI FrontPrep (계속)

1. 담당 업무 (계속)

▾ 지원 환경과 설정 파일 버전 변경 방식 명시

지원 환경을 Node 22.22.1+, Next.js 16 App Router, TypeScript 5, pnpm 10으로 명시했습니다.

독립 앱 또는 pnpm workspace에서 선택한 앱 하나를 설정하도록 했습니다. Pages Router, JavaScript 전용 프로젝트, 다른 패키지 매니저, workspace 전체 자동 설정은 v1에서 제외했습니다.

변경 기록 파일(manifest)을 schema v2로 개선하고, 앱 폴더와 저장소 최상위 폴더에 적용할 설정을 구분했습니다. 기존 manifest도 설정 적용 과정에서 변환하고, 실패하면 되돌릴 수 있도록 했습니다.

▾ 실제 배포 패키지 설치와 생성된 프로젝트 동작 검증

Quality·Tailwind·Test·Git Hooks·CI의 5개 설정 모듈을 제공했습니다. 같은 명령을 다시 실행해도 불필요한 변경이 생기지 않는지 확인했습니다.

저장소 소스만 테스트하지 않고, 실제 npm 배포용 압축 파일(tarball)을 독립된 환경에 설치했습니다. 설치된 CLI 명령과 생성된 프로젝트의 품질 검사·테스트·빌드를 검증했습니다.

@mingyeongbin/frontprep@0.1.0-beta.2와 schema v2 manifest를 RECOVERY-CLIENT에 적용해 실제 사용하는 프로젝트에서도 확인했습니다.

2. 트러블 슈팅 및 성과

▾ 프로젝트마다 다른 기존 설정을 보호하며 변경 적용

문제 상황

CLI를 설치하는 프로젝트마다 설정 파일, lockfile, Git 상태, workspace 구조가 달랐습니다. 파일을 일괄 생성하거나 덮어쓰면 기존 코드와 설정을 망가뜨릴 수 있었습니다.

기존 방식의 한계

변경할 내용을 정하지 않고 바로 수정하면, 중간에 실패했을 때 무엇을 되돌려야 하는지 알기 어려웠습니다. 기존 설정과 충돌할 때의 처리 기준도 필요했습니다.

해결 방법

분석 결과로 변경 계획을 만들고, 적용 직전에 파일이 달라졌으면 중단하도록 했습니다. 실패하면 파일·lockfile·manifest를 복원하고, CLI가 수정해도 되는지 확인되지 않은 파일은 자동으로 합치지 않았습니다.

성과

어떤 파일을 바꾸고, 언제 중단하며, 실패하면 무엇을 복원할지를 명확히 했습니다. 환경이 다른 프로젝트에서도 기존 설정을 확인한 뒤 변경을 적용하도록 했습니다.

▾ 소스 테스트를 넘어 사용자가 설치할 패키지까지 검증

문제 상황

로컬 소스는 정상이어도 이전 빌드 결과가 패키지에 포함되면, 사용자는 최신 수정이 빠진 CLI를 설치할 수 있었습니다.

기존 방식의 한계

소스 코드 테스트만으로는 배포 파일의 CLI 실행 경로(bin), 빌드 결과물, 설치 후 동작까지 확인할 수 없었습니다.

해결 방법

prepack 단계에서 반드시 빌드하도록 했습니다. 실제 npm 배포용 tarball을 독립된 환경에 설치하고, CLI 명령 실행과 생성된 프로젝트의 품질 검사·테스트·빌드를 확인했습니다.

성과

개발 중인 소스뿐 아니라 사용자가 내려받을 패키지를 검사하게 됐습니다. 버전·패키징 문제와 기존 환경에서의 호환성 문제를 배포 전에 확인할 수 있도록 했습니다.

고민한 점

지원 환경을 넓힐 것인가, 검증할 수 있는 범위에 집중할 것인가

여러 라우터와 패키지 매니저를 지원하면 더 많은 프로젝트에서 쓸 수 있지만, 검증하지 못한 환경의 문제는 사용하는 개발자가 떠안게 됩니다. v1은 Next.js 16 App Router·TypeScript·pnpm으로 범위를 제한했습니다. 대신 그 안에서 이전 manifest의 변환, workspace 내 앱 선택, 실패 시 복원이 제대로 동작하는지 확인하는 데 집중했습니다.

성장 스토리

프로젝트마다 반복하던 품질 검사와 협업 설정을 CLI로 만들었습니다. RECOVERY-CLIENT에 beta.2를 직접 적용하면서, 도구를 만드는 입장에서는 놓치기 쉬운 설치 환경과 기존 설정의 차이도 확인했습니다. 이후에는 기능을 늘리기 전에 지원 버전, 이전 설정과의 호환성, 의존성 변경, 실패 시 복구 방법을 먼저 정하게 됐습니다.

AI 대화로 프레젠테이션을 만드는 Snapdeck

1. 담당 업무

▾ AI 대화부터 슬라이드 확인·수정까지 제작 화면 구현

콘텐츠 생성 요청, AI와의 대화, 슬라이드 확인·수정으로 이어지는 화면을 구현했습니다. 채팅 내용과 슬라이드 데이터를 연결해 하나의 제작 흐름으로 사용할 수 있도록 했습니다.

화면 표시, 대화 상태, 비동기 응답 처리, 페이지 상태를 컴포넌트와 커스텀 훅으로 나눠 관리했습니다.

▾ 공통 UI와 시각 테스트 구성, 서버 데이터와 화면 데이터 분리

Figma의 디자인 기준을 토큰과 공통 컴포넌트로 만들었습니다. Storybook·Chromatic을 연결해 팀원이 같은 UI를 확인하고, 변경 전후의 화면을 비교할 수 있도록 했습니다.

TanStack Query로 서버 데이터를 관리하고, 서버 응답을 화면에 맞는 데이터로 변환하는 코드를 분리했습니다. API 형식이 바뀌어도 UI 컴포넌트의 수정 범위를 줄일 수 있도록 했습니다.

2. 트러블 슈팅 및 성과

▾ ChatSection에서 대화 상태와 AI 응답 처리를 분리

문제 상황

대화 이력, 프롬프트 입력, 펼침·접힘, AI 응답 처리와 화면 표시가 하나의 ChatSection에 모여 있었습니다.

기존 방식의 한계

동작 하나를 수정해도 전체 채팅 흐름에 영향이 없는지 확인해야 했습니다. AI 응답 방식이나 외부 데이터 연결을 바꿀 때 UI 코드까지 함께 수정해야 해 기능 확장과 테스트가 어려웠습니다.

해결 방법

대화 턴은 useContentChatTurns, 사용자 입력과 동작은 useContentChatSection, 비동기 응답은 Scheduler 훅에서 처리하도록 나눴습니다. ChatSection에는 컴포넌트를 조합하고 화면을 표시하는 코드만 남겼습니다.

성과

ChatSection을 229줄에서 110줄로 줄였습니다. 대화 상태와 응답 처리 방식을 UI 코드와 분리해 수정할 수 있게 했습니다.

▾ URL 검증을 분리해 다른 화면에서도 쓰는 TextField로 개선

문제 상황

공통 입력 컴포넌트가 Notion·Web URL 검증, 입력 유형 구분, 성공·오류 문구, 화면 표시까지 모두 처리해 다른 입력 화면에서 재사용하기 어려웠습니다.

기존 방식의 한계

URL 검증 조건이나 안내 문구를 바꿀 때 공통 컴포넌트도 수정해야 했습니다. 입력 유형이 늘어날수록 내부 조건 분기도 늘어났습니다.

해결 방법

URL 검증과 상태 판단은 부모 컴포넌트와 별도 유틸리티로 옮겼습니다. TextField는 전달받은 status와 helperText를 표시하고, 접근성 속성을 제공하도록 바꿨습니다.

성과

새 입력 유형을 추가할 때도 TextField 내부를 고치지 않고, 외부에서 검증 규칙과 안내 문구를 지정해 사용할 수 있게 했습니다.

▾ 슬라이드 순서 변경 후 화면과 서버 데이터를 일치시킴

문제 상황

슬라이드를 드래그해 순서를 바꾼 뒤, 화면에 보이는 순서와 서버에 저장된 순서가 달라질 수 있었습니다.

기존 방식의 한계

화면의 상태만 바꾸면 다시 조회하거나 다른 화면으로 이동했을 때 이전 순서가 나타날 수 있었습니다.

해결 방법

순서 변경 API를 호출하고, 서버가 반환한 결과를 TanStack Query 캐시에 반영했습니다. 이후 화면도 이 데이터를 기준으로 표시하도록 했습니다.

성과

순서를 바꾼 직후와 다시 조회한 화면이 같은 서버 데이터를 사용하도록 맞췄습니다.

고민한 점

화면·상태·데이터 변환 코드를 어디까지 나눌 것인가

채팅 로직을 한곳에 모으면 처음에는 흐름을 읽기 쉽지만, 상태나 응답 방식 하나를 바꿔도 UI 코드까지 수정해야 했습니다. 반대로 작은 훅으로 지나치게 나누면 실행 순서를 따라가기 어려웠습니다. 그래서 코드 길이보다 '함께 바뀌는 기능인가'를 기준으로 나눴습니다.

대화 턴은 useContentChatTurns, 입력과 가이드라인 동작은 useContentChatSection, 비동기 응답은 Scheduler 훅에 모았습니다. 서버 응답을 화면 데이터로 바꾸는 일은 Mapper가 맡고, TextField는 URL 검증 결과를 전달받아 표시하도록 했습니다. 각 코드가 무엇을 맡는지 설명할 수 있고, 다른 역할의 코드를 건드리지 않고 수정할 수 있는지를 확인했습니다.

성장 스토리

처음에는 Figma 화면과 인터랙션을 구현하는 데 집중했습니다. 기능이 늘고 API가 연결되자, 동작하는 화면을 만드는 것만으로는 변경에 대응하기 어렵다는 점을 알게 됐습니다.

ChatSection에 모인 대화 상태와 AI 응답 처리를 나눠 229줄을 110줄로 줄였고, URL 검증에 묶여 있던 TextFieldNotionModal을 여러 입력 화면에서 쓸 수 있는 TextField로 바꿨습니다. 서버 응답과 화면 데이터 사이에는 Mapper를 두어 API 변경을 처리할 위치도 명확히 했습니다.

Figma 기반 디자인 토큰과 Storybook·Chromatic을 구성하면서는 제가 맡은 화면뿐 아니라 팀원이 공통 UI를 개발하고 검증하는 방법도 함께 고민했습니다. 이후에는 기능을 추가하기 전에 변경될 부분과 공통으로 유지할 부분을 먼저 구분하게 됐습니다.

대학생 연합 IT 벤처 창업 동아리 SOPT 37기 WEB 파트

2025년 9월~2026년 1월

1. 활동 요약

합동세미나

티켓베이 리디자인 프로젝트에서 기획·디자인 파트와 처음 협업했습니다. 기획서와 디자인을 구현하면서 사용자 흐름과 기술적으로 가능한 범위를 함께 검토했습니다. 디자인 의도를 확인하고, 구현이 어려운 부분에는 대안을 제안하는 소통 방식을 배웠습니다.

솝커톤

하루 안에 서비스를 기획하고 개발·배포했습니다. 모든 기능을 넣기보다 핵심 기능을 먼저 정하고, 역할을 나눠 완성하는 데 집중했습니다.

짧은 일정에서는 무엇을 구현할지뿐 아니라 무엇을 제외할지도 빨리 결정해야 했습니다. 이후 프로젝트에서도 일정과 인력을 고려해 우선순위를 정하는 기준이 됐습니다.

웹 세미나 및 스터디

프론트엔드 기초 세미나와 매주 과제를 통해 부족했던 내용을 정리했습니다. 서로 코드리뷰를 주고받으며, 기능이 동작하는지뿐 아니라 구현한 이유를 설명할 수 있는지도 확인했습니다.

호기심스 스터디에서는 번들러와 빌드 도구를 살펴보며, 개발 환경이 구성되고 코드가 배포 파일로 만들어지는 과정을 학습했습니다.

고민한 점

수정 요청만 하는 리뷰에서, 선택한 이유를 묻는 리뷰로

이전에는 충돌 없이 기능이 동작하면 별도 논의 없이 머지하곤 했습니다. SOPT에서 과제와 스터디의 코드리뷰를 주고받으며, 같은 기능도 여러 방식으로 구현할 수 있고 선택한 이유를 이해하는 과정이 필요하다는 것을 배웠습니다.

처음에는 다른 사람의 코드에 의견을 남기는 것이 부담스러웠습니다. 이후에는 '이렇게 바꾸세요'보다 '왜 이 구조를 선택했나요?'처럼 의도를 묻고 대안을 함께 비교하려고 했습니다. 코드리뷰를 정답을 판정하는 일보다, 서로의 판단 기준을 맞추고 더 나은 선택을 찾는 대화로 받아들이게 됐습니다.

성장 스토리

개인 프로젝트에서는 맡은 기능이 동작하는 데 집중했다면, SOPT에서는 기획·디자인·다른 개발자와 구현 범위와 이유를 공유하는 경험을 쌓았습니다. 코드리뷰에서 제 선택을 설명하고 질문을 받으면서, 제 코드를 다시 읽고 개선점을 찾는 습관도 생겼습니다.

세미나, 스터디, 합동세미나, 솝커톤을 병행하며 일정과 우선순위를 조율했습니다. 시간이 부족할 때는 기능 수를 늘리기보다 핵심 흐름을 완성하는 쪽을 선택했습니다.

동작하는 코드도 다시 점검했습니다. 불필요한 로직과 렌더링 흐름을 살펴보고, 이미지 로딩을 개선하기 위해 WebP 적용도 시도했습니다. 이 경험을 바탕으로 이후 프로젝트에서는 구현한 이유를 설명하고, 함께 작업할 사람이 이해하기 쉬운 코드를 만들려고 노력하고 있습니다.

SIDE PROJECT

활동 요약

숭실 IT 팀 프로젝트 (장려상 수상) (2025.05~2025.11)
Heal-pick (신장질환 환자들을 위한 건강맞춤 서비스)
Github : HeathyPick_FE

IT 벤처 창업 동아리 SOPT (2025.09 ~ 2026.08)
TicketBay (티켓베이 리디자인) / Github : 37-COLLABORATION-WEB-TICKETBAY
바스락 (SNS 밈 공유 서비스) / Github : 37-SOPKATHON-WEB-WEB1
Snapdeck (PPT AI Agent 리디자인) / Github : 38-COLLABORATION-WEB-SNAPDECK
HASHI (일본 맛집 큐레이션 및 예약 서비스) / Gihhub : HASHI-CLIENT

숭실대학교 중앙축구 동아리 SSC (2023.02 ~ 2025.12)
SSC 중앙 축구 동아리 50기 회장 수행 (2024.02~2024.12)
SUFA 리그 무패우승(2025)